

# MPUM Miniprojekt 3 - Raport

Szymon Trofimiec

23 maja 2026

## 1 Wstęp

Dane podzieliłem na zbiór treningowy, testowy, oraz walidacyjny w stosunku 60%/20%/20%.

## 2 Metoda Wektorów Nośnych

Aby wytrenować SVM rozwiążemy problem maksymalizacji marginesu w formie dualnej. Dla zbioru treningowego  $(x^{(i)}, y^{(i)})$ ,  $i \in [n]$  wyznaczamy:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y_i y_j \kappa(x_i x_j) \alpha_i \alpha_j.$$

Pod warunkiem:

- $0 \leq \alpha_i \leq C \quad \forall_i$ ,
- $\sum_{i=1}^n \alpha_i y_i = 0$ .

Gdzie  $C$  to hiperparametr,  $\kappa$  to funkcja jądrowa, a  $\alpha_i$  to mnożniki Lagrange-a. Do aproksymacji maksimum użyjemy algorytmu SMO.

### 2.1 Optymalizacje

Aby umożliwić fine-tuning modelu musiałem znacznie przyspieszyć proces treningowy (przed jedną epoką trenowała się 5 minut, teraz kilka sekund):

1. Na początku obliczam macierz grama - czyli  $\kappa(x_i, x_j)$  dla wszystkich par  $(i, j)$ , gdzie  $\kappa$  to pewna funkcja jądrowa.
2. Zamiast obliczać za każdym razem błąd dla  $i$ -tej i  $j$ -tej współrzędnej obliczamy na starcie wszystkie błędy, a potem aktualizujemy je w czasie  $O(n)$  (złożoność jest taka sama, jak liczenie za każdym razem błędów od nowa, ale w zoptymalizowanej wersji lepiej korzystamy z cache).
3. Dodałem dodatkowy warunek stopu: W pierwszej wersji czekałem aż 5 iteracji pod rząd nie zaktualizujemy żadnego  $\alpha_i$ . Po kilkudziesięciu iteracjach co epoki zmieniały się 2-3  $\alpha_i$ , a wartość funkcji celu wzrastała bardzo nieznacznie. Aby skrócić czas treningu dodałem dodatkowy warunek: Jeśli funkcja celu nie zmienia się wystarczająco mocno to przerywam trenowanie.

### 2.2 Fine-tuning hiperparametrów

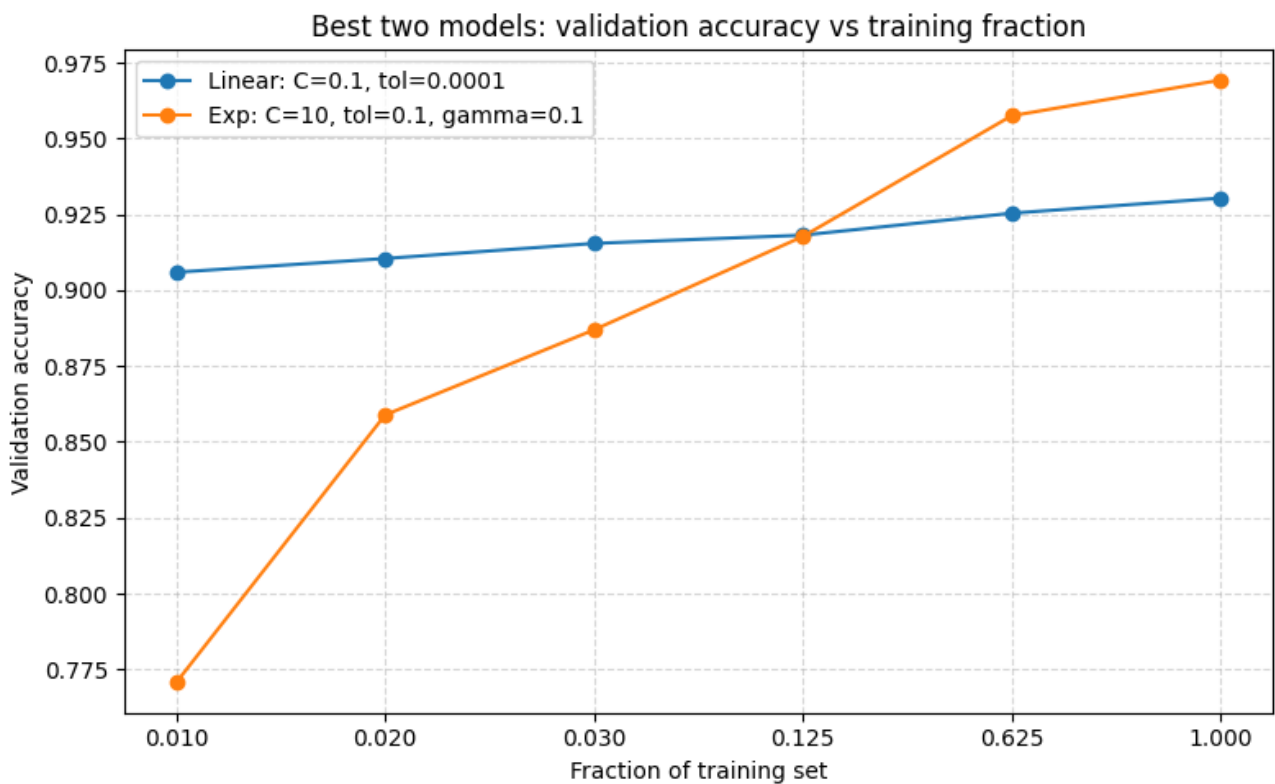
Porównałem wydajności modeli dla różnych hiperparametrów:  $C$  - maksymalnej wartości  $\alpha_i$ ,  $\text{tol}$  - tolerancja na niestabilności numeryczne. Porównałem również dwie funkcje jądrowe: liniową i wykładniczą. Analizując wyniki widać, że SVM z kernelem liniowym osiąga zauważalnie gorsze wyniki niż z kernelem wykładniczym (93% vs 97%). Do końcowej analizy mocy algorytmów wykorzystam 2 modele: Pierwszy z kernelem liniowym,  $C = 0.1$ ,  $\text{tol} = 1e^{-4}$ , a drugi z kernelem wykładniczym  $C = 10$ ,  $\text{tol} = 0.1$ ,  $\gamma = 0.1$ , gdzie  $\gamma$  to parametr funkcji jądrowej.

|                   | 0.010  | 0.020  | 0.030  | 0.125  | 0.625  | 1.000  |
|-------------------|--------|--------|--------|--------|--------|--------|
| C=0.1, tol=0.0001 | 0.9059 | 0.9104 | 0.9154 | 0.9181 | 0.9254 | 0.9303 |
| C=0.1, tol=0.001  | 0.9055 | 0.9104 | 0.9154 | 0.9190 | 0.9263 | 0.9299 |
| C=0.1, tol=0.1    | 0.9082 | 0.9095 | 0.9159 | 0.9186 | 0.9240 | 0.9294 |
| C=1, tol=0.0001   | 0.8584 | 0.8883 | 0.9114 | 0.9204 | 0.9299 | 0.9313 |
| C=1, tol=0.001    | 0.8584 | 0.8874 | 0.9109 | 0.9195 | 0.9281 | 0.9326 |
| C=1, tol=0.1      | 0.8548 | 0.8928 | 0.9037 | 0.9227 | 0.9281 | 0.9326 |
| C=10, tol=0.0001  | 0.7965 | 0.8480 | 0.8865 | 0.9204 | 0.9294 | 0.9317 |
| C=10, tol=0.001   | 0.7960 | 0.8467 | 0.8874 | 0.9204 | 0.9303 | 0.9299 |
| C=10, tol=0.1     | 0.7852 | 0.8526 | 0.8878 | 0.9254 | 0.9276 | 0.9322 |

|                               | 0.010  | 0.020  | 0.030  | 0.125  | 0.625  | 1.000  |
|-------------------------------|--------|--------|--------|--------|--------|--------|
| C=0.1, tol=0.0001, gamma=0.01 | 0.5432 | 0.5432 | 0.5441 | 0.9055 | 0.9168 | 0.9199 |
| C=0.1, tol=0.0001, gamma=0.1  | 0.5432 | 0.5432 | 0.5780 | 0.8281 | 0.9240 | 0.9353 |
| C=0.1, tol=0.001, gamma=0.01  | 0.5432 | 0.5432 | 0.5441 | 0.9064 | 0.9163 | 0.9199 |
| C=0.1, tol=0.001, gamma=0.1   | 0.5432 | 0.5432 | 0.5780 | 0.8281 | 0.9240 | 0.9353 |
| C=0.1, tol=0.1, gamma=0.01    | 0.5432 | 0.5432 | 0.5441 | 0.9091 | 0.9163 | 0.9199 |
| C=0.1, tol=0.1, gamma=0.1     | 0.5432 | 0.5432 | 0.5920 | 0.8304 | 0.9254 | 0.9344 |
| C=1, tol=0.0001, gamma=0.01   | 0.8304 | 0.9204 | 0.9145 | 0.9172 | 0.9267 | 0.9308 |
| C=1, tol=0.0001, gamma=0.1    | 0.7947 | 0.8883 | 0.9028 | 0.9276 | 0.9525 | 0.9620 |
| C=1, tol=0.001, gamma=0.01    | 0.8417 | 0.9204 | 0.9145 | 0.9172 | 0.9267 | 0.9308 |
| C=1, tol=0.001, gamma=0.1     | 0.7938 | 0.8883 | 0.9028 | 0.9276 | 0.9525 | 0.9620 |
| C=1, tol=0.1, gamma=0.01      | 0.8611 | 0.9209 | 0.9145 | 0.9168 | 0.9254 | 0.9308 |
| C=1, tol=0.1, gamma=0.1       | 0.7879 | 0.8892 | 0.9032 | 0.9267 | 0.9516 | 0.9602 |
| C=10, tol=0.0001, gamma=0.01  | 0.8978 | 0.9159 | 0.9118 | 0.9231 | 0.9448 | 0.9444 |
| C=10, tol=0.0001, gamma=0.1   | 0.7698 | 0.8557 | 0.8829 | 0.9186 | 0.9575 | 0.9692 |
| C=10, tol=0.001, gamma=0.01   | 0.8978 | 0.9159 | 0.9118 | 0.9236 | 0.9448 | 0.9444 |
| C=10, tol=0.001, gamma=0.1    | 0.7702 | 0.8557 | 0.8829 | 0.9186 | 0.9575 | 0.9692 |
| C=10, tol=0.1, gamma=0.01     | 0.8942 | 0.9141 | 0.9136 | 0.9249 | 0.9444 | 0.9444 |
| C=10, tol=0.1, gamma=0.1      | 0.7707 | 0.8589 | 0.8869 | 0.9177 | 0.9575 | 0.9692 |

Rysunek 1: Trafność SVM z kernelem liniowym, oraz wykładniczym na zbiorze walidacyjnym dla frakcji zbiorów treningowego i różnych konfiguracji hiperparametrów  $C$  i  $\text{tol}$ .

### 2.3 Moc SVM z kernelem liniowym vs z kernelem wykładniczym



Tutaj lepiej widać różnice w mocy tych dwóch modeli. Warto jednak zaznaczyć, że trening zatrzymywał się przed osiągnięciem zbieżności funkcji celu, więc potencjalnie więcej epok zwiększyłoby trafność.

### 3 Drzewa decyzyjne

W tym programie będziemy budowali drzewo rekurencyjnie: w każdym węźle wybieramy cechę względem której otrzymujemy jak najlepszy podział próbek (jako metryki jakości podziału używam zanieczyszczenia giniego). Na zbiorze walidacyjnym przetestowałem różne konfiguracje hiperparametrów: maksymalnej głębokości drzewa i minimalną wielkość zbioru próbek na której dokonujemy podziału. Wyniki fine-tuningu są następujące:

|                            | 0.010  | 0.020  | 0.030  | 0.125  | 0.625  | 1.000  |
|----------------------------|--------|--------|--------|--------|--------|--------|
| max_depth=3, min_split=2   | 0.8774 | 0.9132 | 0.9132 | 0.9114 | 0.9023 | 0.9019 |
| max_depth=3, min_split=3   | 0.8774 | 0.9132 | 0.9132 | 0.9114 | 0.9023 | 0.9019 |
| max_depth=3, min_split=5   | 0.8774 | 0.9132 | 0.9132 | 0.9114 | 0.9023 | 0.9019 |
| max_depth=3, min_split=7   | 0.8774 | 0.9132 | 0.9132 | 0.9114 | 0.9023 | 0.9019 |
| max_depth=3, min_split=10  | 0.8774 | 0.9132 | 0.9132 | 0.9114 | 0.9023 | 0.9019 |
| max_depth=5, min_split=2   | 0.9014 | 0.9064 | 0.9000 | 0.9159 | 0.9249 | 0.9218 |
| max_depth=5, min_split=3   | 0.9014 | 0.9064 | 0.9000 | 0.9159 | 0.9249 | 0.9218 |
| max_depth=5, min_split=5   | 0.8973 | 0.9064 | 0.9000 | 0.9159 | 0.9249 | 0.9218 |
| max_depth=5, min_split=7   | 0.8973 | 0.9064 | 0.9000 | 0.9154 | 0.9249 | 0.9218 |
| max_depth=5, min_split=10  | 0.8774 | 0.9118 | 0.9000 | 0.9145 | 0.9249 | 0.9218 |
| max_depth=10, min_split=2  | 0.8462 | 0.8824 | 0.8684 | 0.9019 | 0.9290 | 0.9408 |
| max_depth=10, min_split=3  | 0.8431 | 0.8824 | 0.8553 | 0.8946 | 0.9290 | 0.9389 |
| max_depth=10, min_split=5  | 0.8272 | 0.9050 | 0.8716 | 0.9019 | 0.9276 | 0.9371 |
| max_depth=10, min_split=7  | 0.8272 | 0.9064 | 0.8847 | 0.9082 | 0.9294 | 0.9367 |
| max_depth=10, min_split=10 | 0.8774 | 0.9118 | 0.8788 | 0.9014 | 0.9335 | 0.9353 |
| max_depth=20, min_split=2  | 0.8462 | 0.8824 | 0.8716 | 0.9037 | 0.9380 | 0.9525 |
| max_depth=20, min_split=3  | 0.8431 | 0.8824 | 0.8553 | 0.8937 | 0.9358 | 0.9475 |
| max_depth=20, min_split=5  | 0.8272 | 0.9050 | 0.8716 | 0.9009 | 0.9303 | 0.9480 |
| max_depth=20, min_split=7  | 0.8272 | 0.9064 | 0.8847 | 0.9082 | 0.9317 | 0.9480 |
| max_depth=20, min_split=10 | 0.8774 | 0.9118 | 0.8788 | 0.9014 | 0.9376 | 0.9426 |

Rysunek 2: Trafność drzew decyzyjnych na zbiorze walidacyjnym dla frakcji zbioru treningowego i różnych konfiguracji  $max\_depth$  i  $min\_split$ .

Najlepsze drzewa decyzyjne osiągają trafność lepszą niż SVM z linowym kernelem, ale gorszą niż SVM z kernelem wykładniczym. Do końcowej analizy wykorzystam drzewo decyzyjne z parametrami  $max\_depth = 20$ , oraz  $min\_split = 2$ .

### 4 $K$ najbliższych sąsiadów

W tym algorytmie w celu klasyfikacji ze zbioru treningowego wybieramy  $k$  najbliższych próbek - wtedy wynikiem klasyfikacji jest najczęściej występująca klasa spośród tych nich.

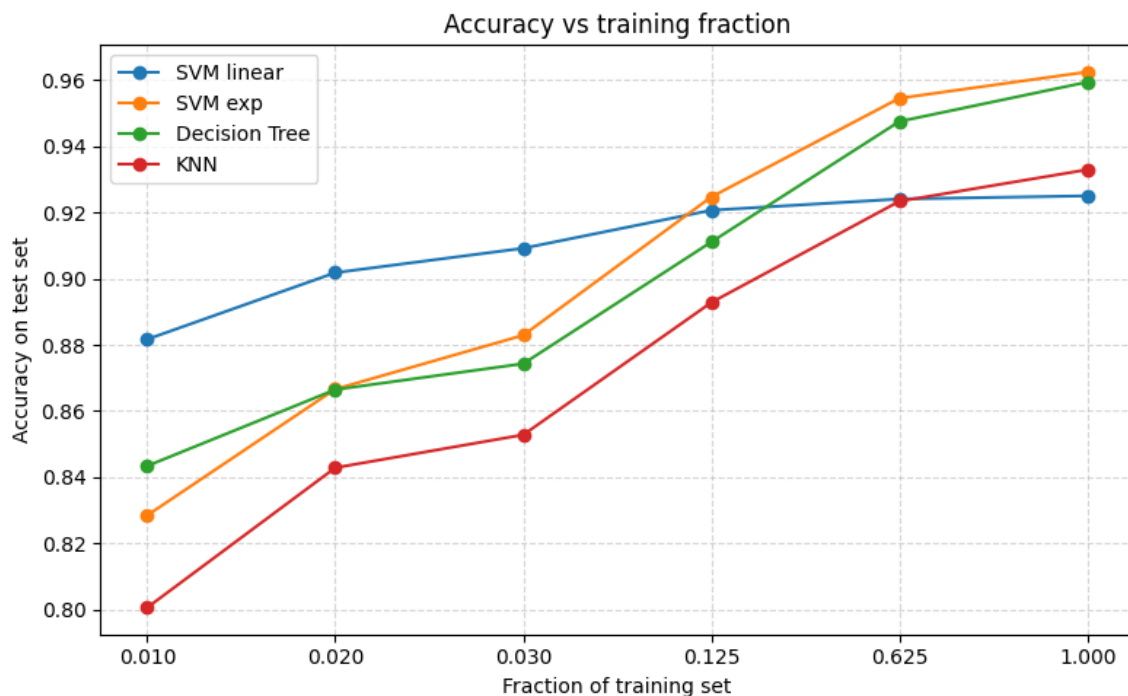
|                | 0.010  | 0.020  | 0.030  | 0.125  | 0.625  | 1.000  |
|----------------|--------|--------|--------|--------|--------|--------|
| n_neighbors=3  | 0.7512 | 0.8327 | 0.8422 | 0.8869 | 0.9335 | 0.9444 |
| n_neighbors=5  | 0.7657 | 0.8548 | 0.8630 | 0.9068 | 0.9331 | 0.9444 |
| n_neighbors=7  | 0.7829 | 0.8625 | 0.8706 | 0.9032 | 0.9335 | 0.9448 |
| n_neighbors=10 | 0.8290 | 0.8512 | 0.8566 | 0.8978 | 0.9294 | 0.9389 |
| n_neighbors=15 | 0.8014 | 0.8711 | 0.8761 | 0.9028 | 0.9290 | 0.9358 |
| n_neighbors=20 | 0.8137 | 0.8725 | 0.8756 | 0.8973 | 0.9227 | 0.9308 |

Rysunek 3: Trafność algorytmu knn na zbiorze walidacyjnym dla frakcji zbioru treningowego i różnych wartości  $k$ .

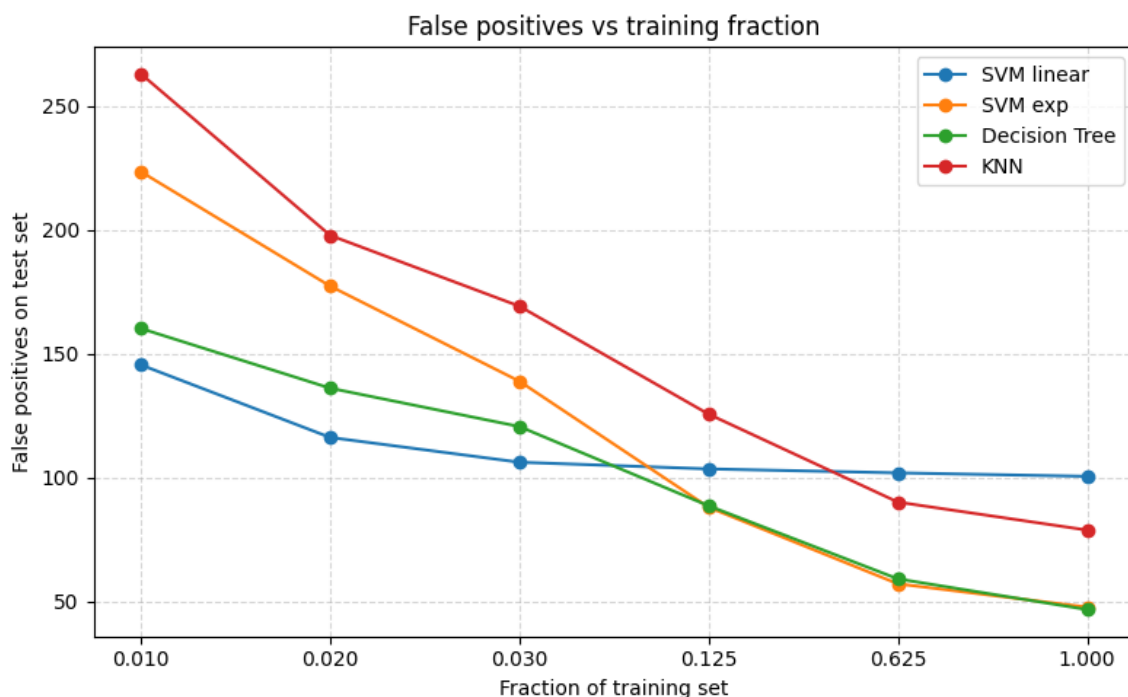
KNN osiąga odrobinę gorszy wynik, niż drzewa decyzyjne. W końcowej analizie wykorzystam algorytm dla  $k = 7$ .

## 5 Porównanie wydajności modeli

Przetestowałem wybrane modele wytrenowane na frakcjach zbioru treningowego. Po uśrednieniu 10 iteracji wyniki są następujące:



W wykrywaniu phishingu jednak nie tylko accuracy jest ważne - chcemy również minimalizować liczbę false positives:



Zaskakująco na zbiorze treningowym drzewa decyzyjne sprawdzają się prawie tak dobrze jak SVM z kernelem wykładniczym - być może wynika to z prostoty zbioru danych. Zauważmy również, że mimo niskiej trafności SVM z kernelem liniowym stabilizuje się już dla 3% zbioru treningowego - dla pozostałych nie wiemy czy cały zbiór treningowy to wystarczająco danych by osiągnąć maksymalny wynik. Dla prawie wszystkich modeli accuracy jest proporcjonalne do liczby false positives, jednak drzewa decyzyjne widocznie lepiej minimalizują ich liczbę od pozostałych modeli.